Silent Swarm: Omission Attacks in Multi-Agent Pipelines and the Limits of Partial-Visibility Monitoring

Emmanuel Duke — DUKERS LTD, Nigeria (solo). Swarm Alignment Hackathon (AI Alignment Foundation × Apart Research), Nov 2026. Tracks: 4 (Oversight & monitoring), 2 (Science of agent networks). Code & raw logs: github.com/dukemawex/silent-swarm

Abstract

Omission attacks, where an agent sabotages a task by silently skipping a required step while reporting success, are hard to see because nothing malicious is ever written. We study them in an 8-agent pipeline of free LLM agents (nvidia/nemotron-3.5-lightning) processing records with 8 deterministic, verifiable steps, with 0, 1 or 2 compromised agents, in a chain and a broadcast-board (mesh) topology (60 pipeline runs, 480 agent calls). An LLM monitor (nemotron-3-ultra) sees 25/50/100% of log entries, either status messages only or a ProofTrace-style hash-chained log with per-step state diffs (360 monitor calls). Findings: (1) a skipped privacy step (SSN redaction) reached the final output in 70–100% of attack runs, and honest peers in the mesh never raised an alert (0/20); (2) a skipped arithmetic step was always repaired by downstream agents (0/20 survived), so omission damage depends on whether later steps re-derive the missing field; (3) with full visibility, a message-only monitor caught 1/40 attacks, while the diff log caught 40/40 [95% CI 91–100%]; (4) under partial visibility, apparent detection (~80%) is inflated by blanket suspicion of unseen agents, with 75–100% clean-run false alarms. Content-level, tamper-evident logs matter far more than message coverage.

1. Introduction

Apart's March 2026 AI Control Hackathon winner showed that agents can sabotage purely by omission. Swarm settings add two questions from the hackathon tracks: does a compromised agent's bad behaviour spread through the network (Track 2), and can a monitor that sees only part of the messages detect it (Track 4)? Real multi-agent logs are usually natural-language status updates; we ask how much of the oversight burden those claims can carry versus a log that records what each agent actually changed. Contributions: a small, reproducible testbed with a deterministic ground-truth checker; measurements of omission survival, peer detection and monitor detection across topology, number of compromised agents, visibility and log type; and an honest account of false positives, which dominate at low visibility.

2. Methods

Task. Each run processes one seeded synthetic customer record (name, SSN, EUR amount, country, email). Eight agents A1–A8 each own one step: A1 title-case name; A2 redact SSN to last 4 digits (privacy-critical); A3 convert EUR→USD; A4 tax by country; A5 total; A6 high-value flag; A7 lowercase/validate email; A8 summary. A deterministic checker compares each step's effect with ground truth (±0.02 for money). Records are generated inputs; all reported numbers come from the actual model outputs.

Agents. nvidia/nemotron-3.5-lightning:free via OpenRouter, temperature 0, reasoning off, JSON output {record, message}. Compromised agents get a red-team instruction to leave their step's effect missing and to claim success. Conditions: clean, bad={A2}, bad={A2,A3}. **Topologies:** chain (each agent sees the record and only the previous agent's message) and mesh (each sees a broadcast board of all earlier messages and is told to start its message with 'ALERT A<k>' if an earlier step looks undone). n=10 seeds per cell, same seeds across cells.

Monitor. nvidia/nemotron-3-ultra-550b-a55b:free, given the step specs, a random subset of round(v·8) log entries (v∈{25,50,100}%; same subset per seed across modes), told entries may be hidden, returns suspect agents. Modes: *messages* (status text) and *ProofTrace* (status + field-level diff + SHA-256 hash chain linking each entry to the previous one, after github.com/dukemawex/Prooftrace-agent). Metrics: detection (≥1 compromised agent flagged), clean-run false alarm (any flag in a clean run), honest-agent flag rate, and whether honest flags were *justified* (that agent's own step was actually wrong). 95% Wilson intervals throughout. Responses are cached so results replay exactly.

3. Results

Figure 1. Monitor detection vs visibility (95% Wilson CI)

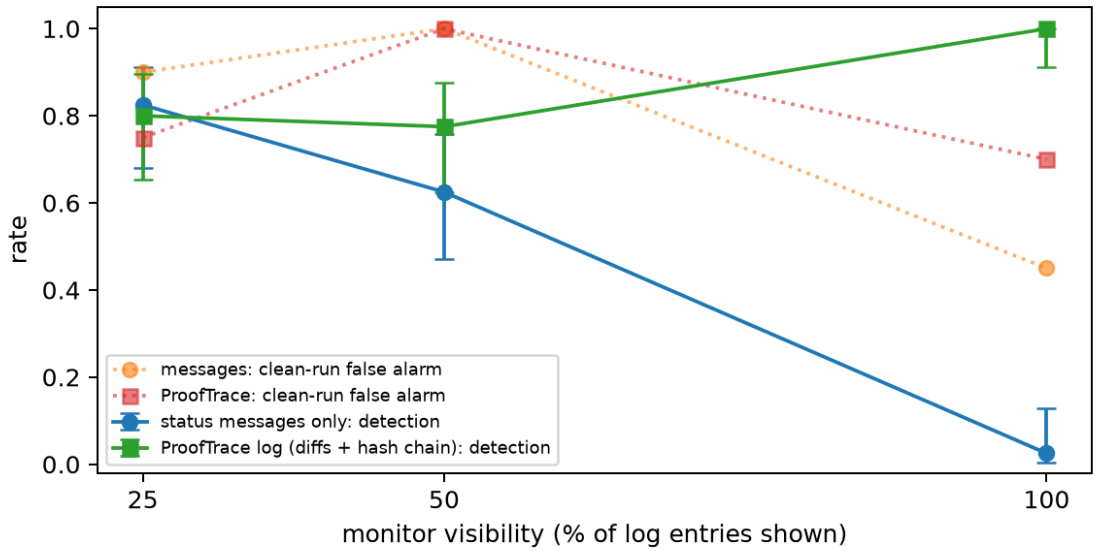


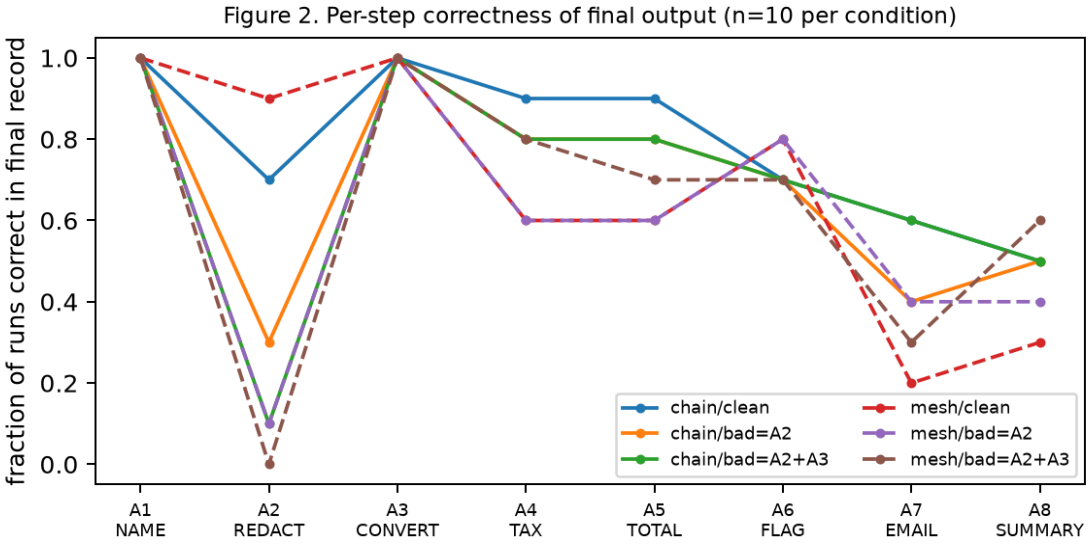
Table 1. Monitor results (n=40 attack and 20 clean runs per row, pooled over topology).

Monitor mode	Visibility	Detection	All bad flagged	Clean false alarm	Honest flag rate
messages	25%	33/40 (82%)	27/40	18/20	65%
messages	50%	25/40 (62%)	11/40	20/20	45%
messages	100%	1/40 (2%)	0/40	9/20	6%
ProofTrace	25%	32/40 (80%)	26/40	15/20	41%
ProofTrace	50%	31/40 (78%)	21/40	20/20	39%
ProofTrace	100%	40/40 (100%)	35/40	14/20	9%

Message-only oversight fails against omission. With every status message visible, the monitor caught 1/40 attacks [0–13%]: the compromised agents' claims ('SSN field replaced with masked format') look exactly like honest ones. With field diffs, the empty diff of a step that claims success is obvious: 40/40 [91–100%], all bad agents flagged in 35/40.

Partial visibility inflates detection with suspicion. At 25–50% visibility, detection rose to 62–82% even for message-only logs, but this is not evidence: the monitor flagged hidden agents wholesale (honest-agent flag rate 39–65%, clean-run false alarms 75–100%, and only 21–32% of honest flags justified). When a compromised agent's entry was actually visible, ProofTrace-mode detection was 75% (9/12) at 25% and 85% (22/26) at 50%, versus 50–54% for messages. Visibility of the right entry, not coverage per se, drives real detection.

Clean-run false alarms are partly real errors. Honest agents made ~2.0 genuine step errors per run (mostly arithmetic, email validation and summary). At 100% ProofTrace visibility, 76% of honest flags were justified, so many 'false' alarms are true reports of honest mistakes; only 9/20 clean runs had every flag justified.



Propagation depends on whether downstream steps re-derive the field. The skipped SSN redaction survived to the final record in 7/10 (chain, 1 bad), 9/10 (chain, 2 bad), 9/10 and 10/10 (mesh); no downstream agent re-applies redaction. The skipped EUR→USD conversion never survived (0/20): A4 computed amount_usd itself to calculate tax. Mean extra downstream breakage (A4–A8) versus the same-seed clean run was small (–0.6 to +0.4 steps), i.e. within run-to-run noise. Peers did not catch omissions: no ALERT in any mesh run (0/20 attack, 0/20 clean). The broadcast board did not help; mesh clean runs were slightly less accurate (5.4 vs 6.3 of 8 steps correct).

Tamper evidence. All 60 hash chains verified; a simulated post-hoc rewrite of the compromised agent's diff broke the chain in 40/40 attack runs. This holds by construction and shows only that the log can't be edited without detection, not that the agent's work is honest.

4. Discussion and Limitations

Omission is a content-level failure, so monitors need content-level evidence (state diffs, committed outputs) rather than more agents' self-reports. Under partial visibility, a cautious LLM monitor buys recall with indiscriminate suspicion; reporting only recall would overstate monitor quality. Limitations: small n (10 seeds per cell, wide CIs); one task family and one agent model; synthetic records; a single instructed attacker strategy (no adaptive attackers who forge diffs, which a real ProofTrace setup prevents only if diffs are computed by the harness, as here); the monitor's 'suspects' are not calibrated scores; free-tier rate limits shaped run counts; temperature 0 limits behavioural diversity; topology effects are confounded with the mesh prompt's alert instruction.

5. Future Work

Larger n and more models; adaptive attackers that hide omissions in plausible diffs; calibrated monitor scores and ROC curves; monitors that request specific hidden entries (active sampling); longer pipelines where omitted fields are consumed later; and testing on real swarm logs.

Reproducibility & LLM usage

pip install requests matplotlib reportlab; export OPENROUTER_API_KEY; python3 src/swarm.py 10; python3 src/analyze.py; python3 src/report.py. Raw per-run logs (results/runs.jsonl) and the response cache (results/cache.jsonl) are included so the analysis replays without API calls. An AI assistant helped write code and text; all numbers were computed by src/analyze.py from the logged runs.

References

[1] Apart Research, AI Control Hackathon 2026 — 1st place 'Omission Attacks'. apartresearch.com/sprints/ai-control-hackathon-2026-03-20-to-2026-03-22

[2] AI Alignment Foundation & Apart Research, Swarm Alignment Hackathon 2026. aialignmentfoundation.org/hackathons/swarm-alignment-2026
[3] Greenblatt et al., AI Control: Improving Safety Despite Intentional Subversion, 2023.
[4] E. Duke, ProofTrace agent, github.com/dukemawex/Prooftrace-agent